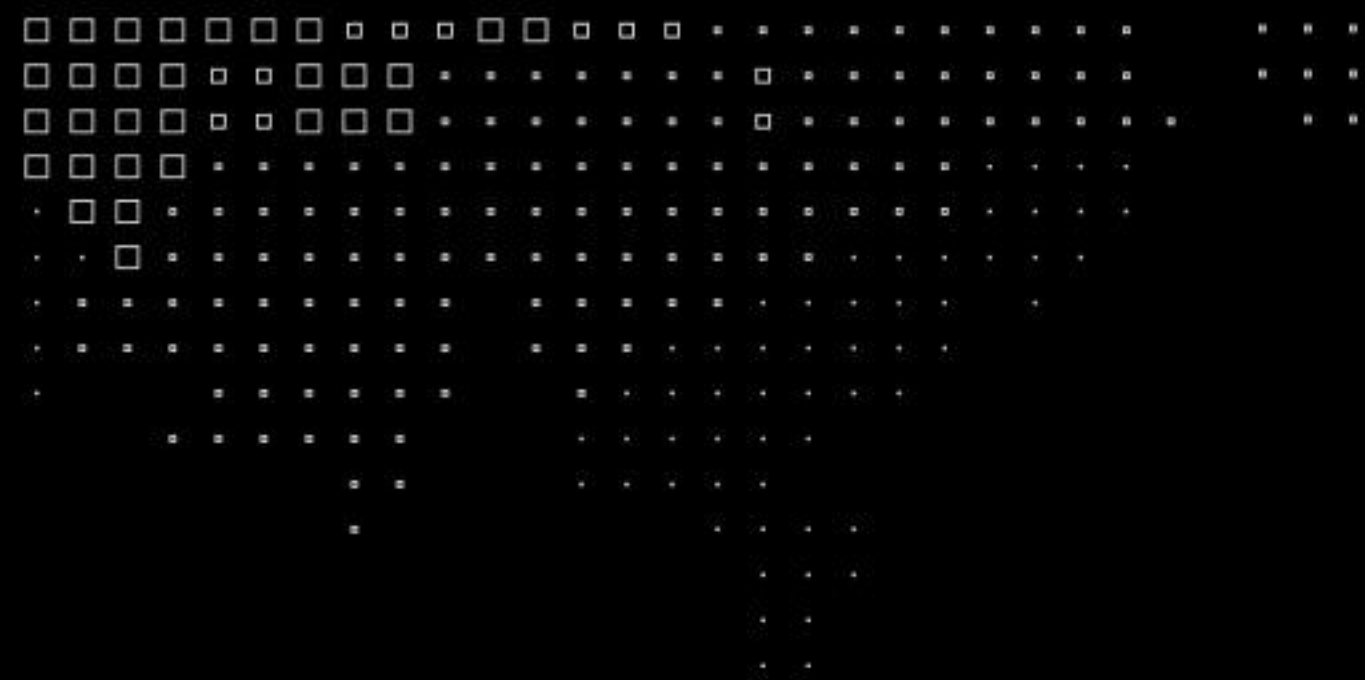
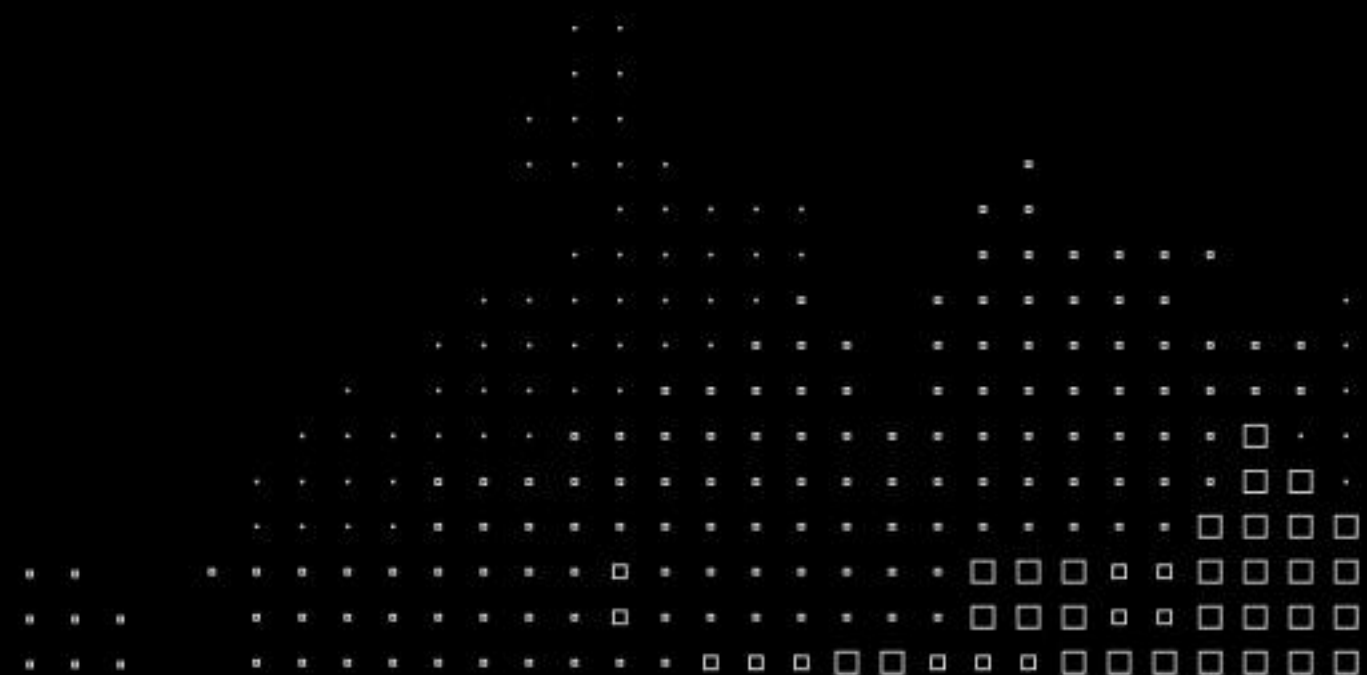




Ciência da Computação



DSA

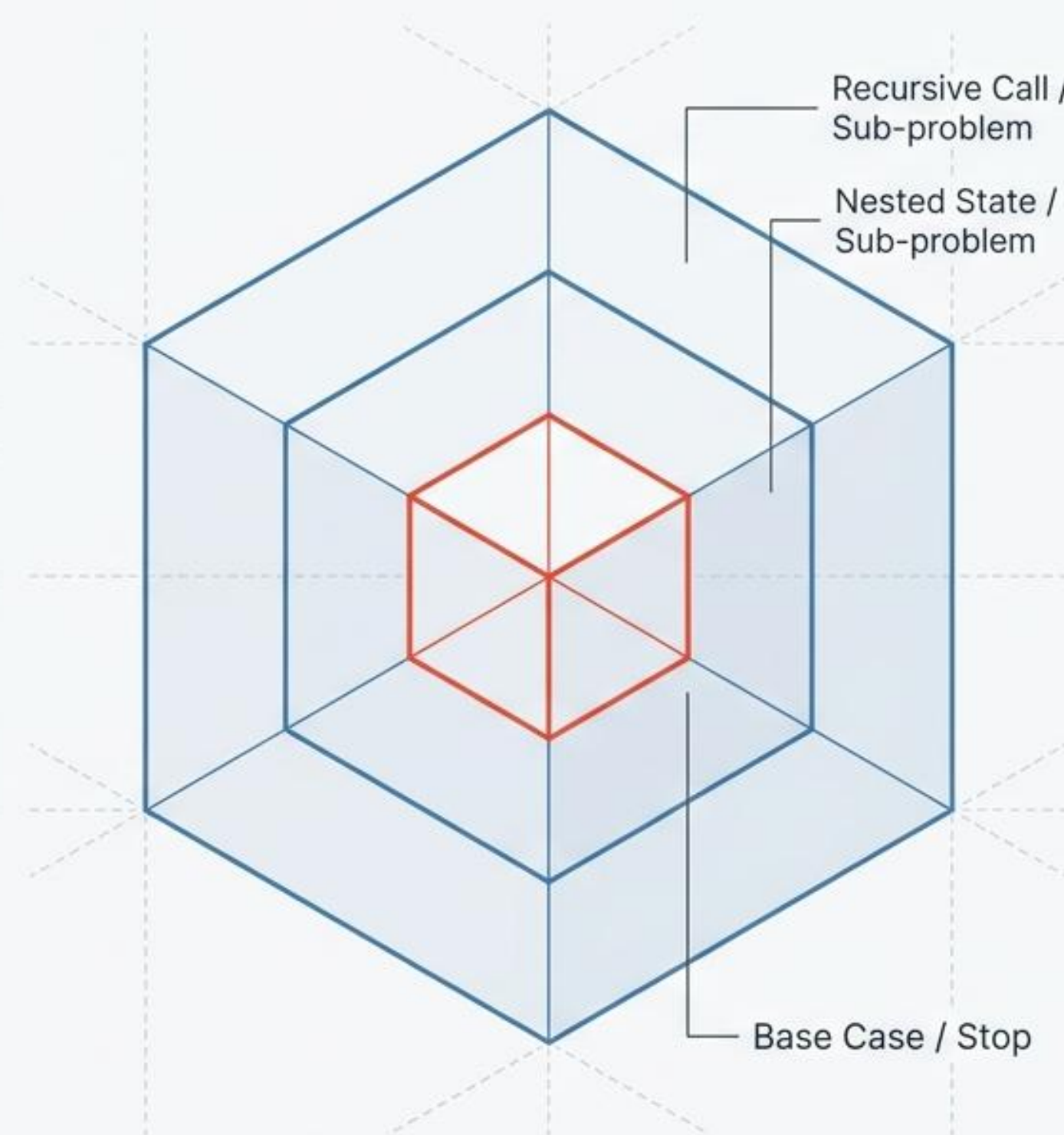
Data Structures and Algorithms

Professor Álvaro Gonçalves
Email: profalvaro.goncalves@fiap.com.br

DATA STRUCTURES & ALGORITHMS

Recursividade

Do caso-base à Busca Binária.



A grande ilusão sobre a recursão

Recursividade é apenas outra forma de fazer um loop (for/while). Falso. O verdadeiro valor da recursividade não é substituir laços iterativos, mas permitir que um problema complexo seja descrito em termos de versões menores de si mesmo.



O paradigma de pensamento dita a abordagem

Iterativo

A Pergunta Central

Qual é o próximo passo?

O Estado

Variáveis atualizadas continuamente (ex: soma += i).

O Risco

Loop infinito se a condição nunca for atingida.

Recursivo

A Pergunta Central

Se eu soubesse a solução para uma versão menor, como usaria isso agora?

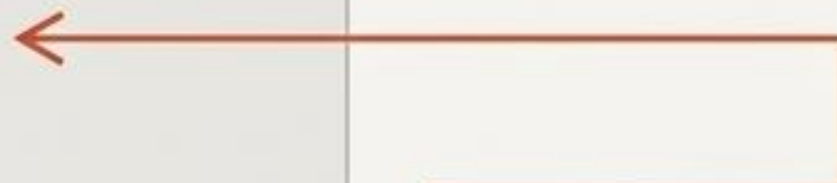
O Estado

Parâmetros isolados na Call Stack.

O Risco

RecursionError se a profundidade esgotar a memória antes do caso-base.

Dissecando a anatomia da solução

```
def contagem(n):  
    if n == 0:   
        return  
    print(n)  
     contagem(n - 1)
```

O Caso-Base [O Freio].
Define quando não
é necessária uma
nova chamada.

O Passo Recursivo [O Encolhimento].
Transforma o problema atual em uma
versão fisicamente menor.

Ter um caso-base não garante a parada

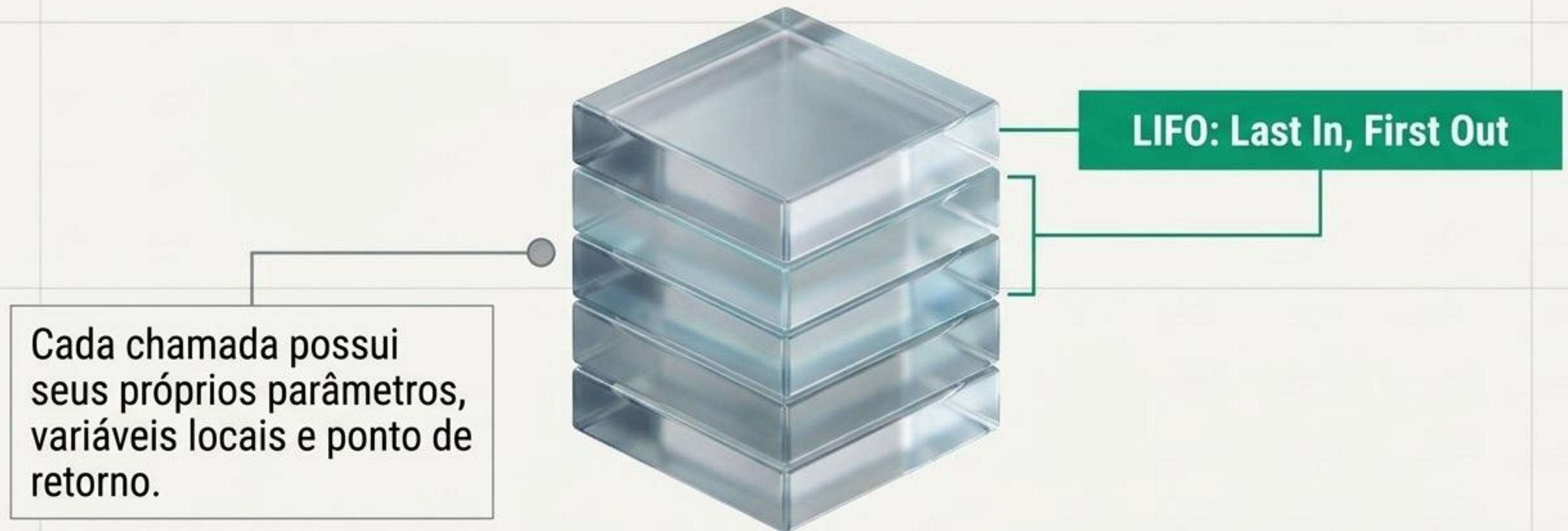
Uma função recursiva precisa caminhar fisicamente em direção ao caso-base a cada chamada. Se o estado não mudar, a função cai em um **buraco negro infinito**.

```
def contagem(5):  
    ...  
    contagem(5)
```

n nunca muda. O caso-base (0) nunca será alcançado.

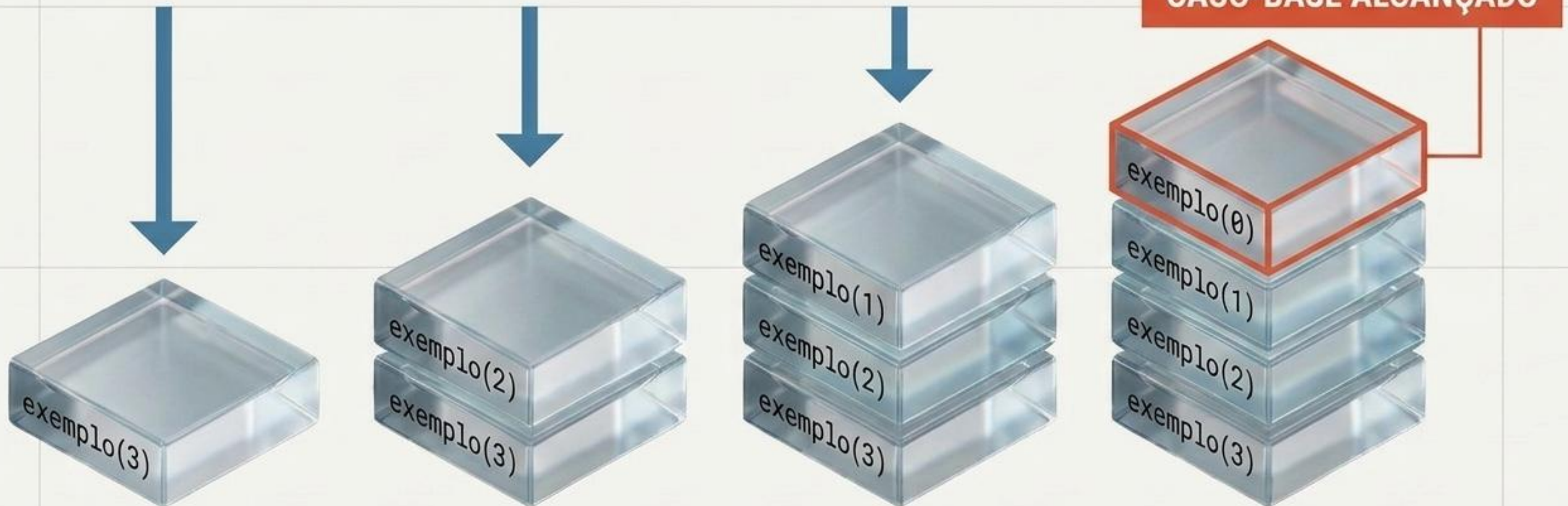
O que acontece com as chamadas que não terminaram?

Quando `contagem(5)` chama `contagem(4)`, a primeira chamada não termina. Ela entra em pausa e fica aguardando na pilha de execução (Call Stack).



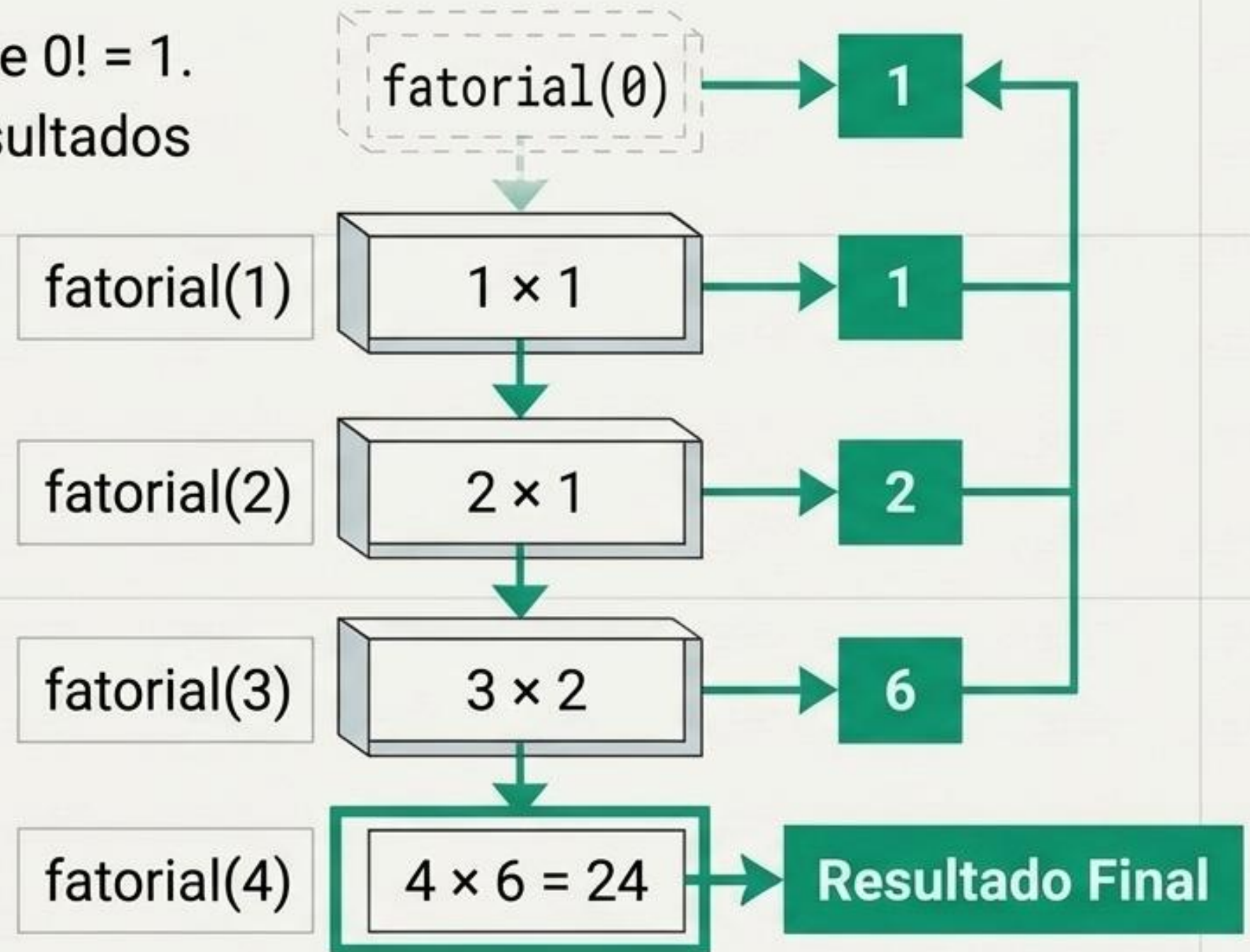
A fase de descida empilha os subproblemas

O problema original é decomposto e empilhado até que a condição de parada absoluta seja encontrada.



A fase de retorno engatilha o efeito dominó

Matematicamente: $n! = n \times (n - 1)!$ e $0! = 1$.
Apenas após atingir o fundo, os resultados começam a subir e multiplicar.



Recursividade não é sinônimo de maior velocidade

O principal valor da recursão é expressivo. Para operações simples, loops iterativos são frequentemente melhores.

Iterativo

```
def soma_iterativa(n):  
    total = 0  
    for i in range(1, n + 1):  
        total += i  
    return total
```

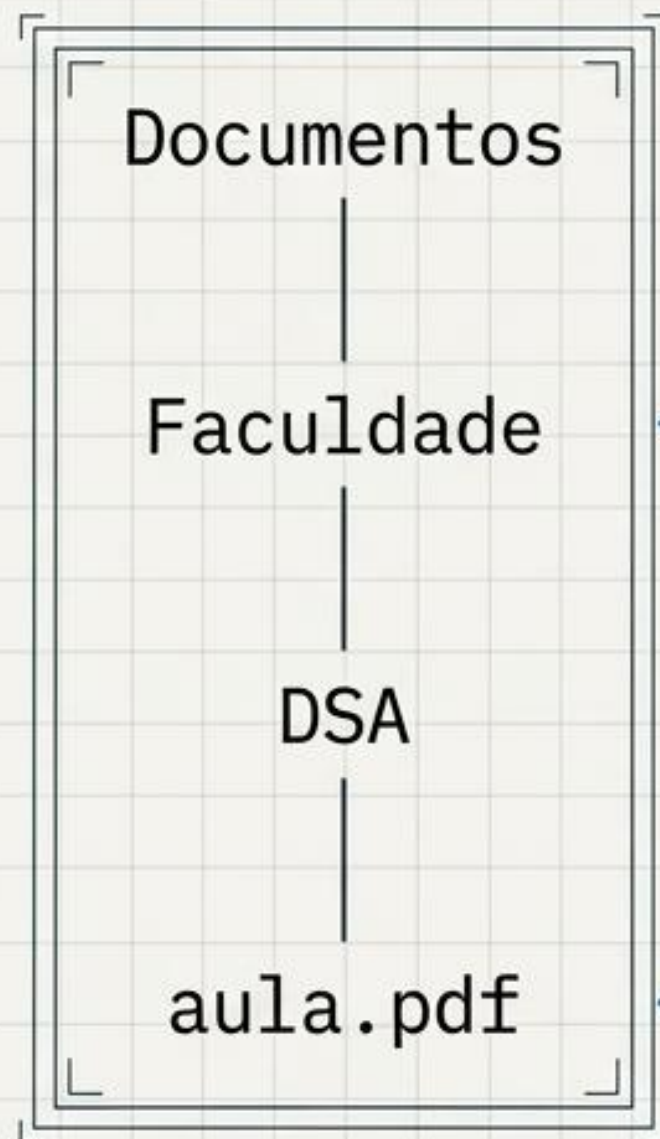
Recursivo

```
def soma_recursiva(n):  
    if n == 0:  
        return 0  
    return n + soma_recursiva(n-1)
```

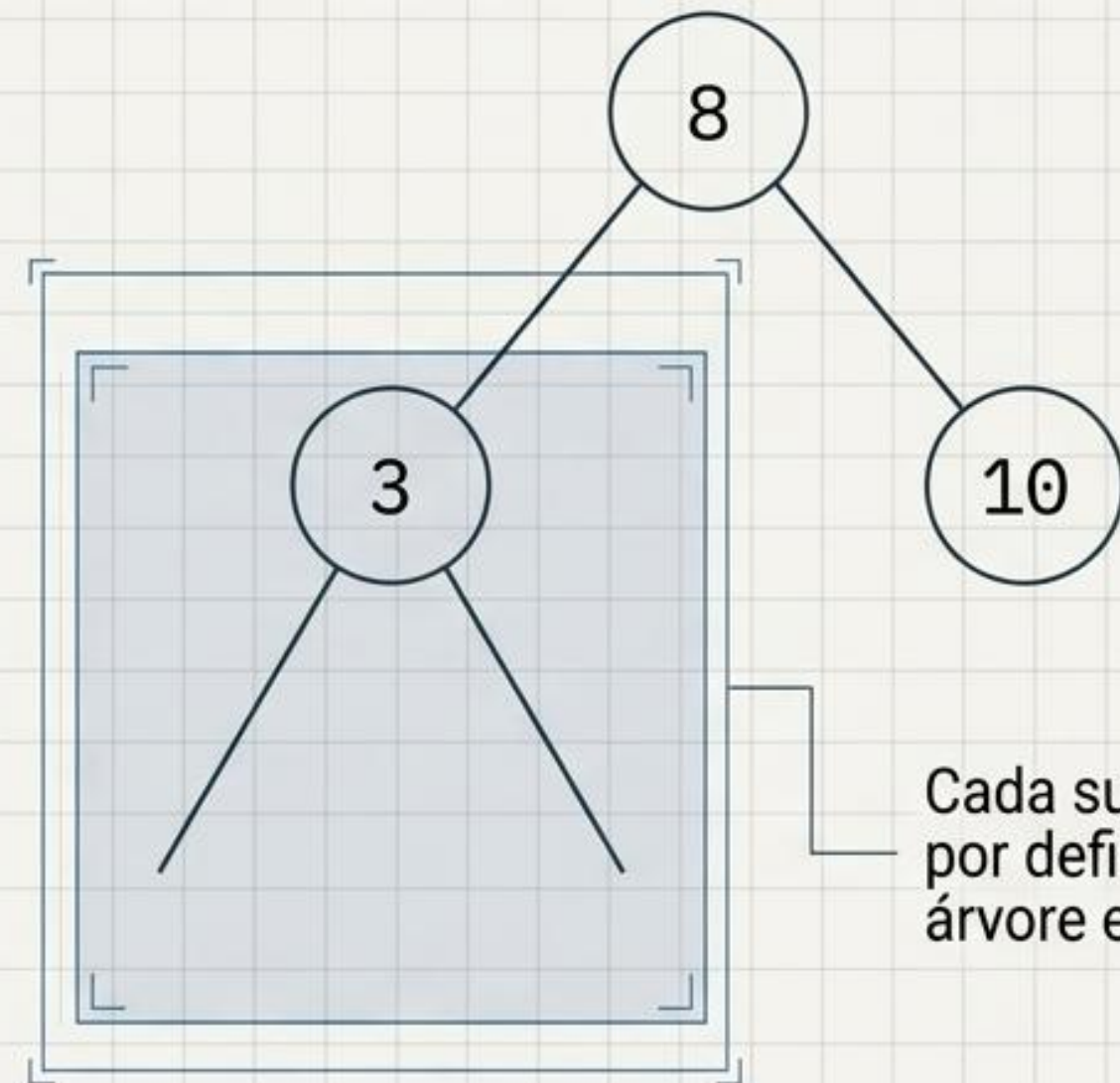
Característica	Iterativo	Recursivo
Custo Oculto	Baixo	Alto (Overhead de função)
Uso de Memória	Constante $O(1)$	Linear na Pilha $O(n)$
Risco de Crash	Baixo	Alto (<code>RecursionError</code>)

Os habitats naturais da recursividade

A recursividade brilha quando a própria estrutura dos dados contém subestruturas do mesmo tipo.



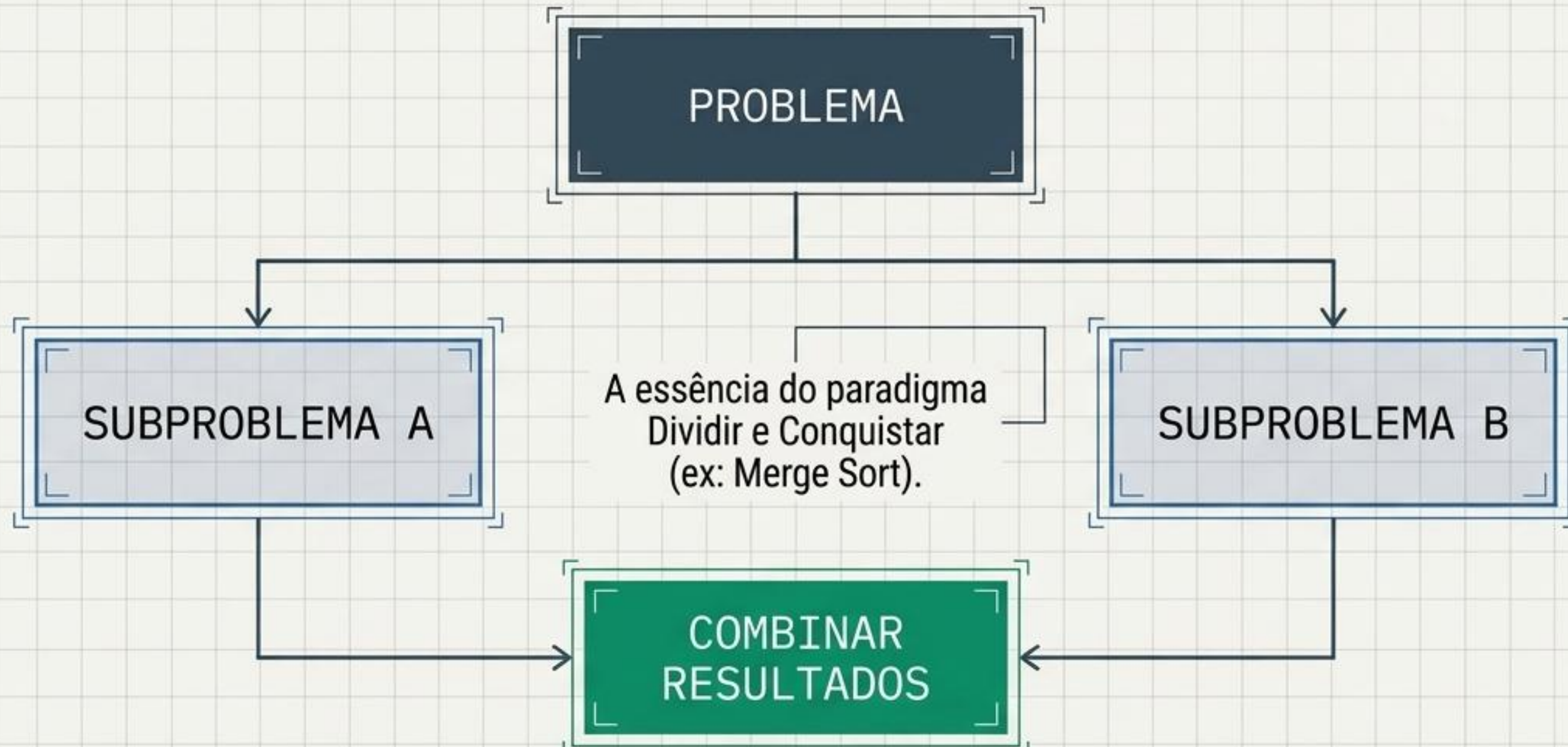
Estrutura recursiva de diretórios:
Uma pasta dentro de uma pasta é conceitualmente a mesma estrutura.



Cada subárvore é, por definição, uma árvore em si.

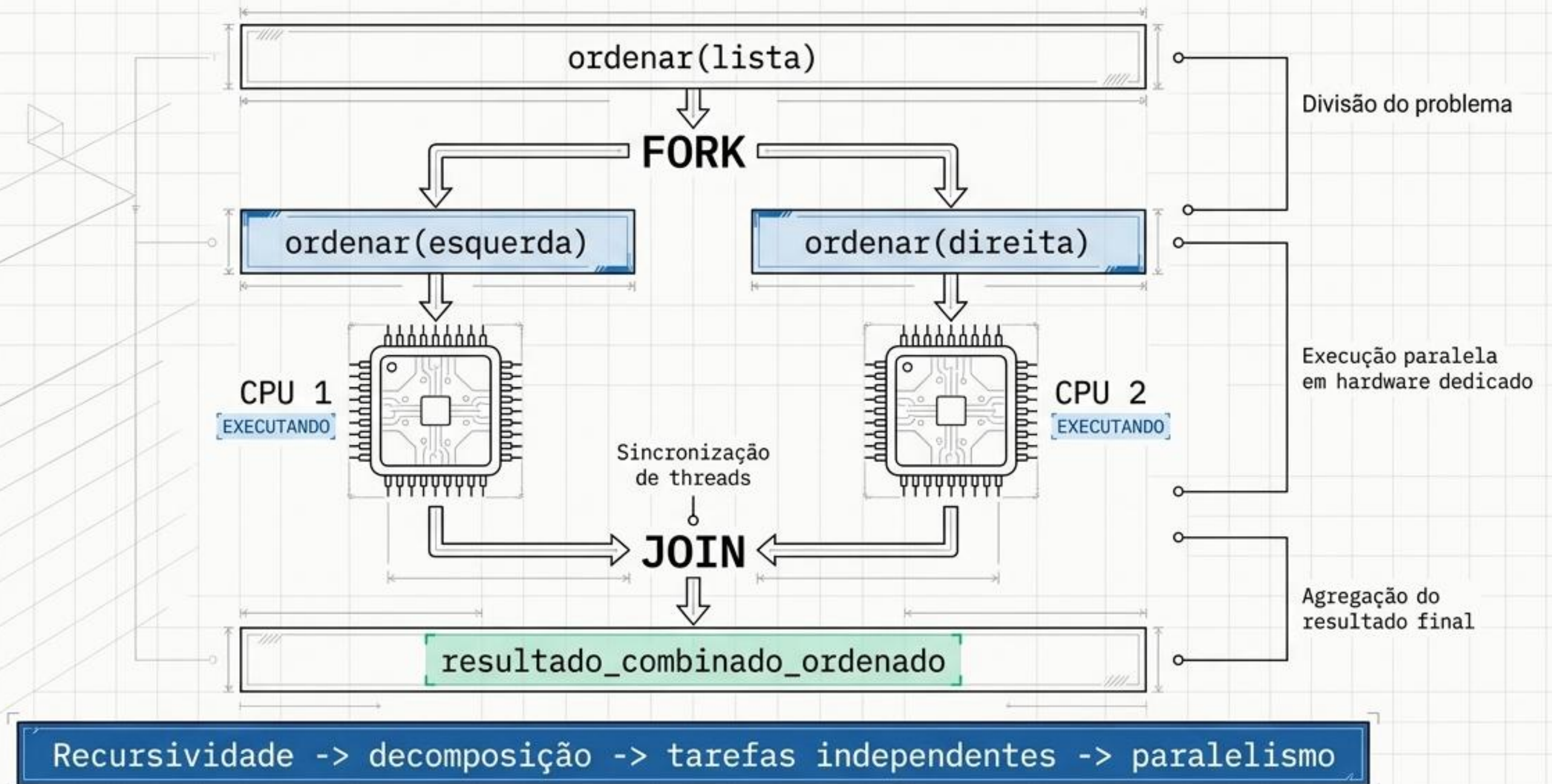
Escalando para múltiplos subproblemas

Até agora, vimos uma cadeia linear de chamadas. Mas o verdadeiro poder surge quando o problema pode ser dividido em duas ou mais frentes simultâneas.



O padrão Fork/Join e o nascimento do paralelismo

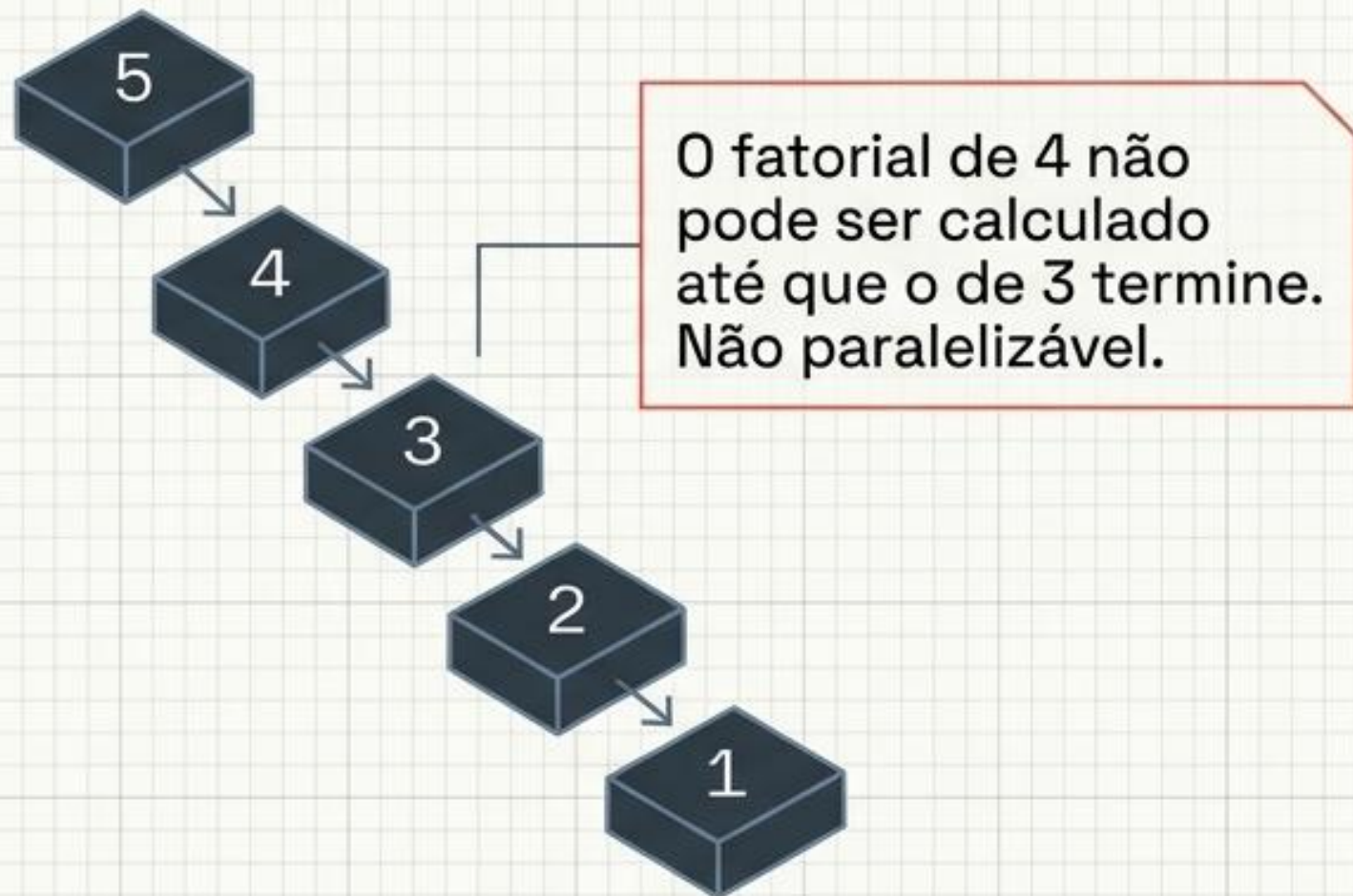
Se houver recursos computacionais adequados, tarefas decompostas de forma independente podem ser executadas simultaneamente.



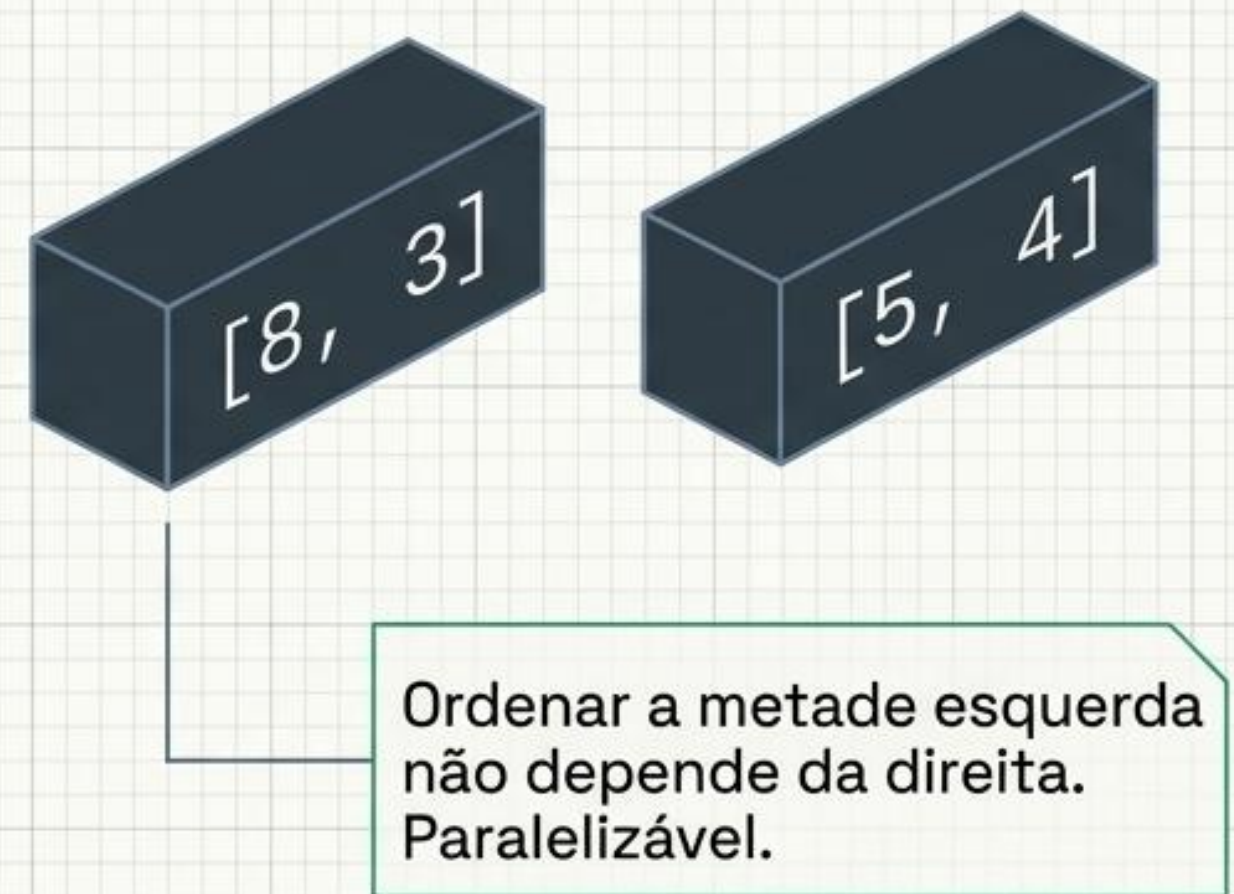
Alerta: Paralelismo exige independência

O simples fato de um algoritmo ser recursivo não cria paralelismo magicamente. A possibilidade depende da estrutura de dependências, não da chamada de função.

Dependência Sequencial

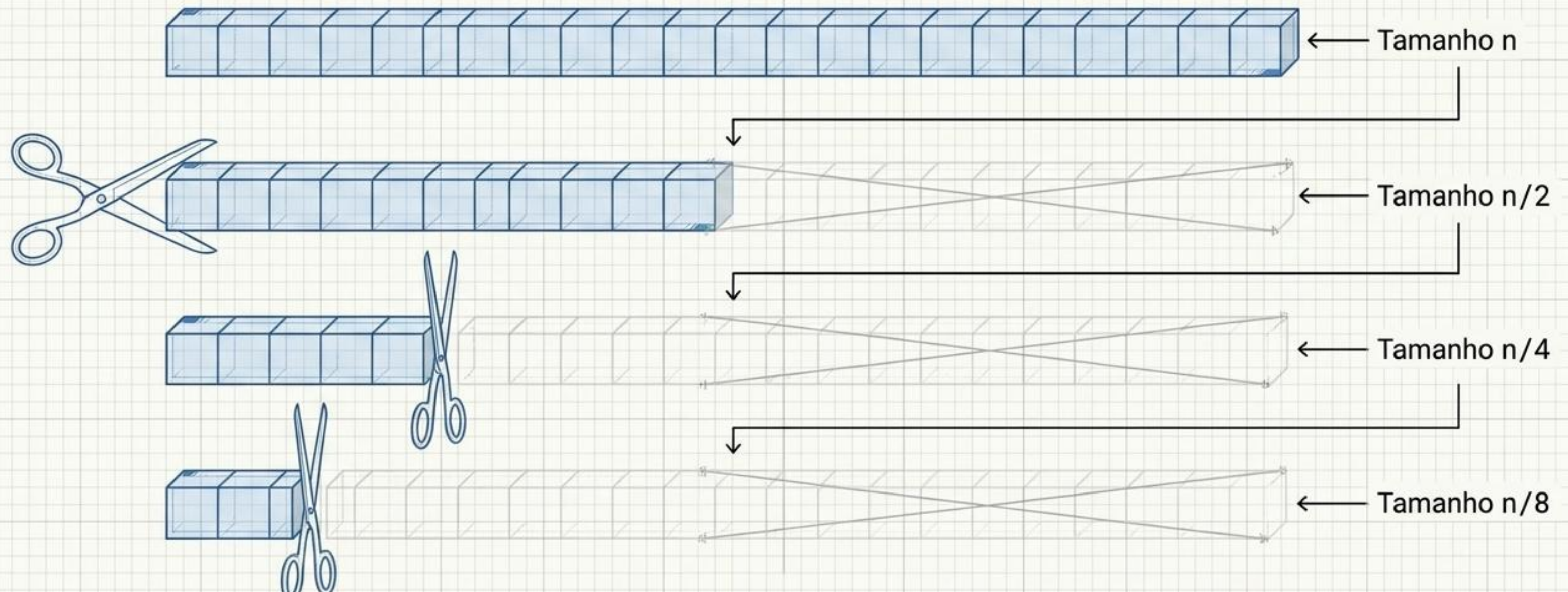


Independência



Estudo de Caso: O algoritmo de redução perfeita

A Busca Binária é o epítome da recursão eficiente porque já possui a estrutura nativa de redução drástica: a cada etapa, metade do universo é descartada.



As 3 perguntas para a Busca Binária

1. Caso-Base (O Freio)

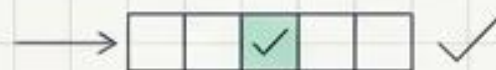
O que acontece se o vetor acabar?



```
if inicio > fim:  
    return -1
```

2. Condição de Sucesso (O Alvo)

O que acontece se o valor estiver no meio?



```
if lista[meio] == valor:  
    return meio
```

3. Redução do Problema (A Tesoura)

Para qual metade devo ir?



```
if valor < lista[meio]:  
    busca na esquerda.  
else:  
    busca na direita.
```

O código construído e anotado

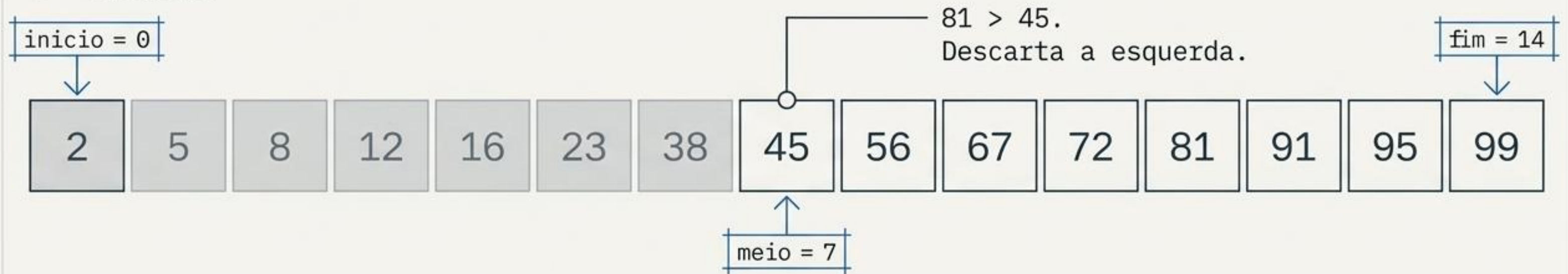
```
def busca_binaria(lista, valor, inicio, fim):  
    if inicio > fim: return -1  
    meio = (inicio + fim) // 2  
    if lista[meio] == valor: return meio  
    if valor < lista[meio]:  
        return busca_binaria(lista, valor, inicio, meio - 1)  
    return busca_binaria(lista, valor, meio + 1, fim)
```

A função transforma o problema original em uma única chamada menor, atualizando apenas os ponteiros de inicio ou fim.

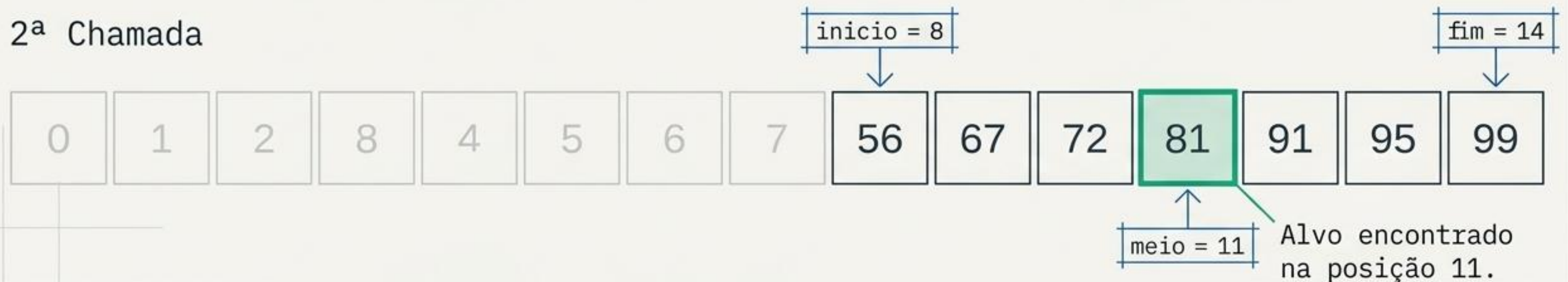
Rastreando a execução na memória

Buscando o valor 81 em um vetor ordenado de 15 posições.

1ª Chamada



2ª Chamada

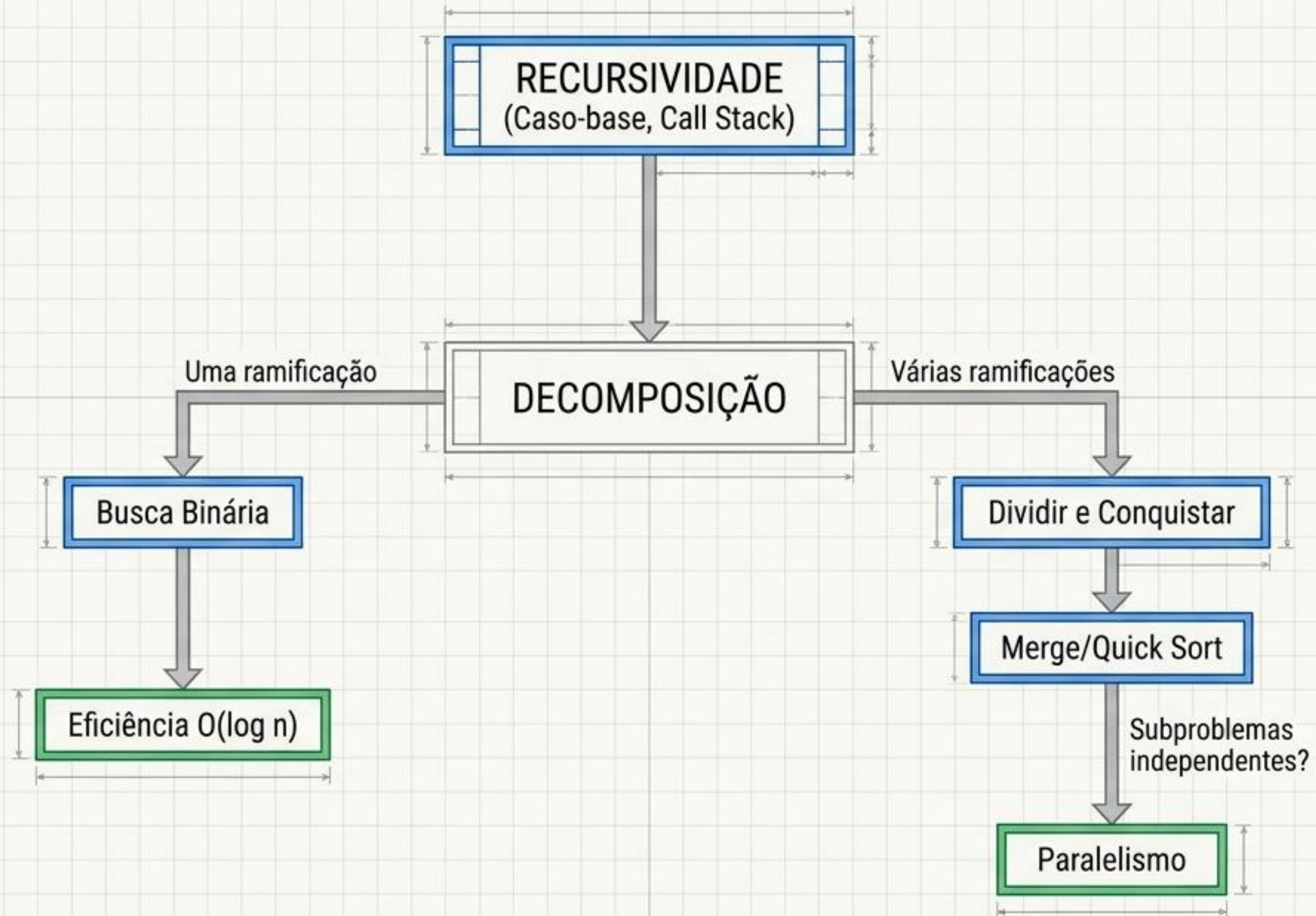


O diagnóstico de eficiência da Busca Binária

A eficiência de $O(\log n)$ vem da redução do vetor pela metade, não do uso da recursividade. A versão recursiva custa mais memória.

Métrica	Iterativa	Recursiva
Tempo de Execução	$O(\log n)$	$O(\log n)$
Espaço Auxiliar	$O(1)$	$O(\log n)$
Usa Call Stack?	Não	Sim
Exige vetor ordenado?	Sim	Sim

A árvore genealógica da decomposição



O checklist do desenvolvedor

Use estas 5 perguntas para analisar, debugar ou criar qualquer função recursiva.

☐

Qual é o caso-base absoluto?

☐

Qual é a chamada recursiva exata?

☐

Como os parâmetros garantem que o problema fica menor?

☐

O que a função faz com o valor quando a chamada retorna?

☐

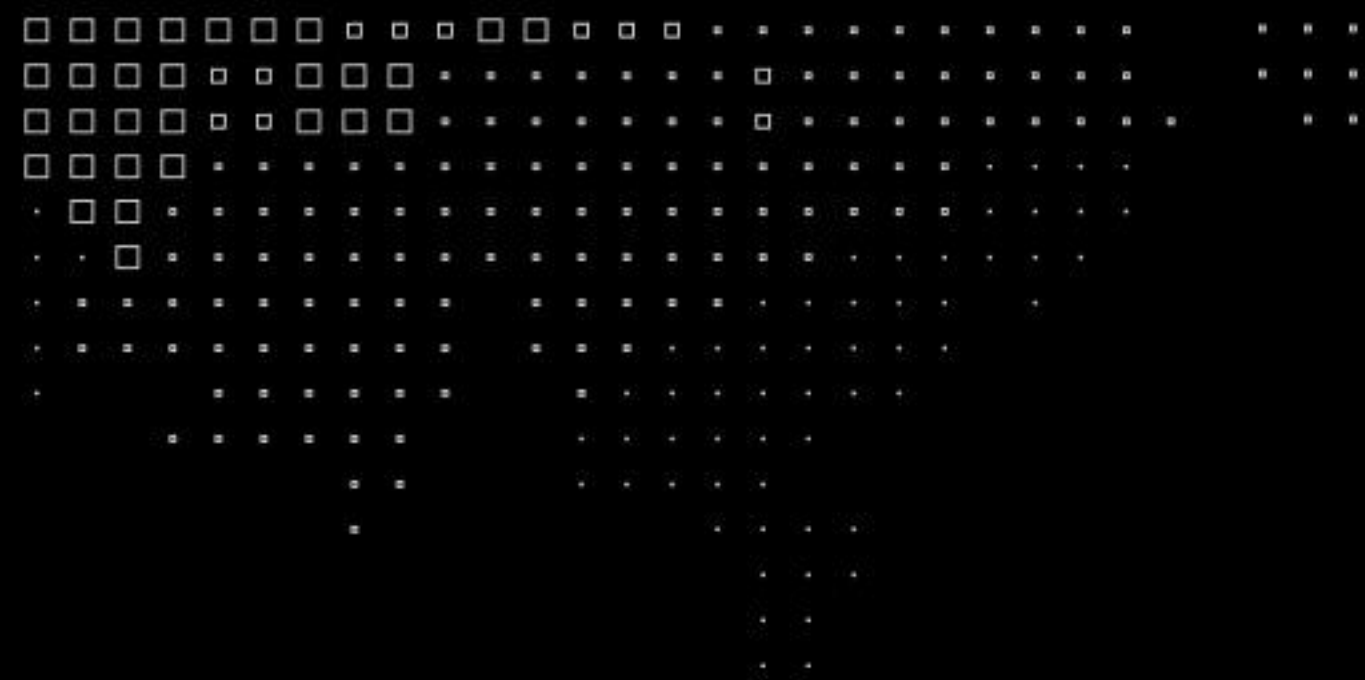
Existem subproblemas que poderiam ser processados de forma independente?

O próximo nível de complexidade

A Busca Binária é elegante porque escolhe apenas UMA das metades e ignora o resto. Mas em problemas reais de ordenação e processamento massivo de dados, não podemos descartar nada.

**E se precisarmos resolver
recursivamente as DUAS
metades ao mesmo tempo?**

Próximo Módulo: Dividir e Conquistar, Merge Sort e Quick Sort.



Copyright © **2026** Prof. Álvaro Gonçalves

Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proibido sem o consentimento formal, por escrito, do Professor Álvaro Gonçalves.

